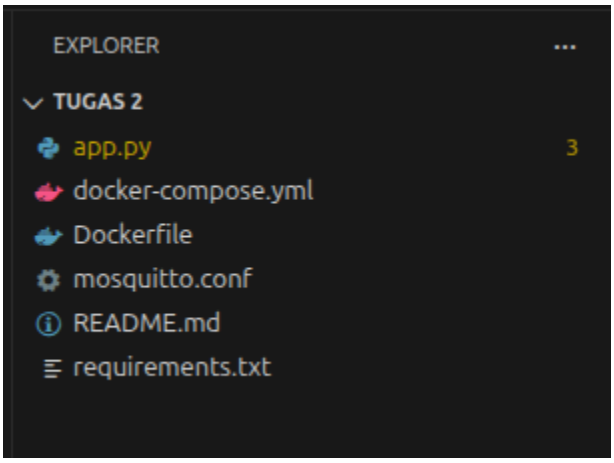
Sistem Paralel dan Terdistribusi - A
Tugas 2 - Simulasi Interaktif Model Komunikasi dalam Sistem Terdistribusi
Abdullah Adiwarman Wildan - 11231001

1. Arsitektur Sistem

Sistem ini dirancang sebagai platform simulasi interaktif yang berjalan di atas framework Streamlit dan menggunakan Mosquitto (MQTT Broker) sebagai pusat distribusi pesan. Seluruh ekosistem dikelola menggunakan Docker Compose untuk mengorkestrasi dua layanan utama dalam jaringan virtual yang sama. Secara lengkap, Komponen arsitektur sistem nya yaitu: (1) Application Node dengan Streamlit yang dimana bertindak sebagai antarmuka pengguna, pengirim pesan atau publisher, dan penerima respons. (2) Broker Node dengan Mosquitto yang bertindak sebagai perantara dalam model asinkron yang menangani pengiriman pesan ke berbagai topik. (3) MQTT Client Logic yang mengimplementasi library paho-mqtt untuk komunikasi asinkron yang nyata antar kontainer. (4) Network Latency Simulator yang menggunakan blocking logic untuk mensimulasikan keterlambatan jaringan.



Gambar 1.1 Arsitektur sistem

2. Model Komunikasi

Sistem ini mengimplementasikan dua jenis model komunikasi fundamental dengan karakteristik yang bertolak belakang untuk menunjukkan dinamika sistem terdistribusi:

a. Model Request-Response (Synchronous): Model ini dirancang untuk mensimulasikan protokol komunikasi sekuensial di mana klien mengirimkan permintaan dan tertahan (blocking) hingga menerima balasan penuh dari server. Logika ini memperlihatkan ketergantungan kuat (tight coupling) antar komponen dalam sistem.
b. Model Publish-Subscribe (Asynchronous): Model ini memanfaatkan protokol MQTT di mana pengirim (publisher) menyiarkan pesan ke Broker tanpa harus menunggu respons

dari penerima (subscriber). Implementasi ini menunjukkan sifat loose coupling yang memungkinkan sistem tetap responsif meski beban data meningkat.

3. Desain

3.1. Elemen Antarmuka

- Bar: Memungkinkan pengguna untuk memilih model komunikasi yang ingin disimulasikan.
- Slider: Digunakan untuk mengatur jumlah pesan dan tingkat latensi jaringan secara fleksibel.
- Tombol Start: Tombol utama untuk memulai proses simulasi berdasarkan parameter yang telah diatur.
- Output Panel: Panel visual yang metrik performa.

Gambar 3.1 Desain Antarmuka

Gambar 3.2 Desain Antarmuka

3.2. Alur Penggunaan

1. Pertama, pilih model komunikasi yang diinginkan melalui menu bar.
2. Tentukan volume data dengan mengatur jumlah pesan yang akan dikirim.
3. Atur tingkat latensi untuk mensimulasikan hambatan pada jaringan sistem terdistribusi.
4. Klik tombol "Jalankan model" untuk menjalankan proses komunikasi.
5. Amati hasil log dan metrik yang muncul pada panel output untuk melakukan analisis.

4. Dokumentasi Deployment (Docker Compose)

Seluruh proses deployment dikemas secara utuh menggunakan Docker untuk memastikan bahwa environment simulasi tetap konsisten pada berbagai perangkat. Instruksi teknis untuk menjalankan sistem adalah sebagai berikut:

1. Orkestrasi Layanan: Pengguna menjalankan perintah docker-compose up --build pada terminal untuk memicu pembuatan image dan pengaktifan jaringan internal.
2. Manajemen Kontainer: Docker Compose akan menginisialisasi kontainer aplikasi dan broker Mosquitto secara otomatis berdasarkan konfigurasi yang telah didefinisikan.
3. Akses Antarmuka: Setelah kontainer berstatus running, simulasi dapat diakses secara interaktif melalui browser pada alamat http://localhost:8501.

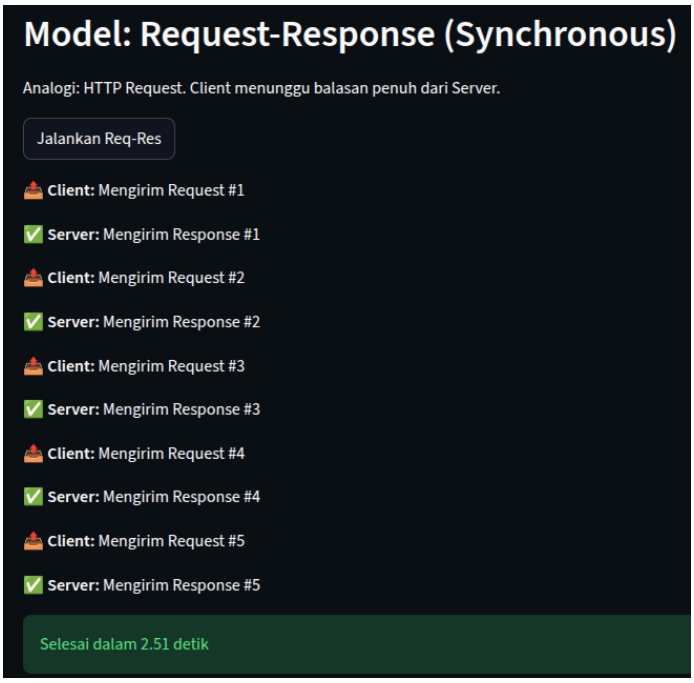
```
wildan@wildan-Predator-PH517-61:~/Documents/ITK/Sister/Tugas 2$ docker compose up --build
WARN[0000] /home/wildan/Documents/ITK/Sister/Tugas 2/docker-compose.yml: the attribute `version` is o
 to avoid potential confusion
WARN[0000] Docker Compose is configured to build using Bake, but buildx isn't installed
[+] Building 0.1s (11/11) FINISHED
 => [app internal] load build definition from Dockerfile
 => => transferring dockerfile: 578B
 => [app internal] load metadata for docker.io/library/python:3.9-slim
 => [app internal] load .dockerignore
 => => transferring context: 2B
 => [app 1/5] FROM docker.io/library/python:3.9-slim
 => [app internal] load build context
 => => transferring context: 3.28kB
 => CACHED [app 2/5] WORKDIR /app
 => CACHED [app 3/5] COPY requirements.txt .
 => CACHED [app 4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [app 5/5] COPY . .
 => [app] exporting to image
 => => exporting layers
 => => writing image sha256:5cbbbf6e83144dec8d379a2cf7e9f33acd505a8af496c7e06ceeaec397ebe78a
 => => naming to docker.io/library/tugas2-app
 => [app] resolving provenance for metadata file
[+] Running 4/4
 ✔ app                          Built
 ✔ Network tugas2_default       Created
 ✔ Container mosquitto_broker   Created
 ✔ Container streamlit_simulasi Created
Attaching to mosquitto_broker, streamlit_simulasi
mosquitto_broker    | 1776287527: Info: running mosquitto as user: mosquitto.
```

Gambar 3.1 Proses Deployment

5. Hasil Benchmarking (Simulasi)

Analisis performa dilakukan untuk membandingkan efisiensi antara kedua model komunikasi. Berdasarkan data yang dikumpulkan selama simulasi dengan parameter latensi 0.5s dan volume 5 pesan, didapatkan hasil sebagai berikut:

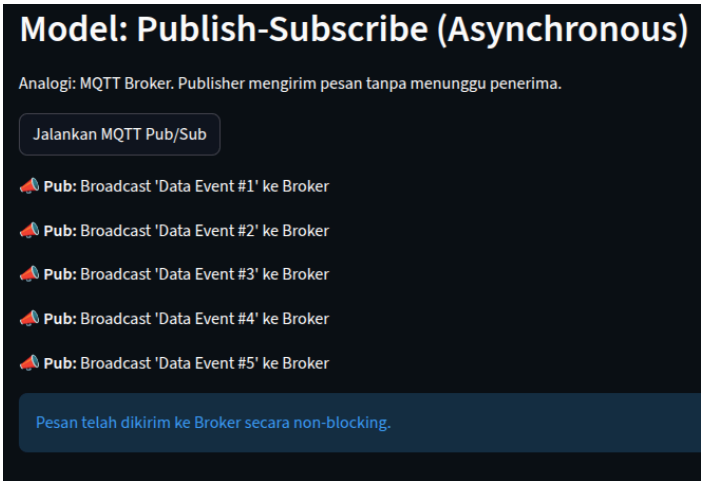
1. Model Synchronous



Gambar 4.1 Request-Response

Didapatkan total waktu eksekusi sekitar 2.5s dengan aliran data sinkron dan kapasitas through put rendah dan blocking

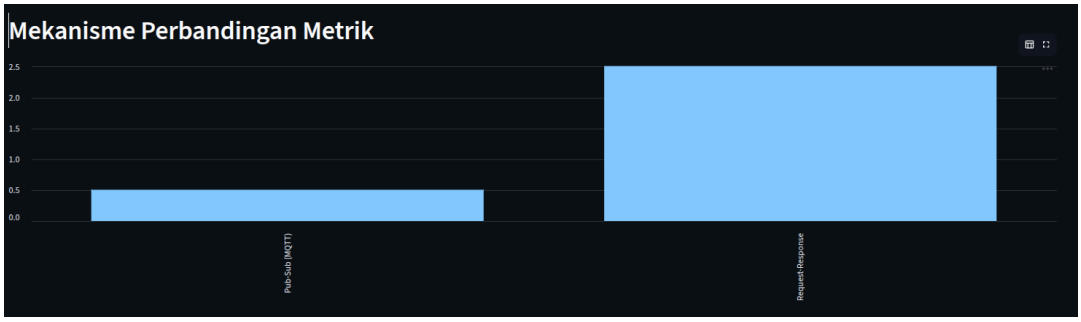
2. Model Asynchronous



Gambar 4.2 Publish-Subscribe

Didapatkan total waktu eksekusi sekitar 0.5s dengan aliran data paralel dan kapasitas through put tinggi dan non-blocking

3. Analisis Perbandingan kedua model



Gambar 4.3 Metrik perbandingan

Model Pub/Sub menunjukkan kapasitas throughput yang jauh lebih baik karena sisi pengirim tidak terikat oleh kecepatan pemrosesan di sisi penerima. Meskipun model Request-Response memiliki waktu eksekusi yang lebih lama, model ini memberikan kepastian status pengiriman yang sangat penting untuk integritas data. Secara keseluruhan, model Pub/Sub menawarkan skalabilitas horizontal yang lebih unggul dalam arsitektur sistem terdistribusi modern.

6. Penutup

Simulasi ini berhasil mendemonstrasikan pertukaran krusial dalam sistem terdistribusi dengan membandingkan model Request-Response (Synchronous) dan Publish-Subscribe (Asynchronous). Meskipun model Synchronous memberikan kepastian status pengiriman yang sangat penting sehingga cocok untuk aplikasi kritis seperti sistem pembayaran, API backend, dan autentikasi dimana sifat *blocking*-nya menghasilkan *throughput* yang lebih rendah. Sebaliknya, model Asynchronous, dengan aliran data paralel dan kapasitas *throughput* yang tinggi, terbukti jauh lebih unggul dalam skalabilitas horizontal. Hal ini menjadikannya pilihan utama untuk skenario modern seperti IoT (sensor monitoring), sistem notifikasi, dan *streaming* data. Dengan demikian, simulasi ini memberikan gambaran nyata bagaimana pemilihan model komunikasi memengaruhi performa dan skalabilitas sistem dalam arsitektur terdistribusi nyata.